Министерство образования Республики Беларусь
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Кафедра электронных вычислительных машин

ОТЧЕТ
О ЛАБОРАТОРНОЙ РАБОТЕ № 2
“ Программирование контроллера прерываний”
Вариант 20

Выполнил ст. гр. 450502 Е.Ю. Новицкий

Проверил асс. каф. ЭВМ А.В. Марзалюк

Минск 2026

Постановка задачи:

Написать резидентную программу выполняющую перенос всех векторов аппаратных прерываний ведущего и ведомого контроллера на пользовательские прерывания. При этом необходимо написать обработчики аппаратных прерываний, которые будут установлены на используемые пользовательские прерывания и будут выполнять следующие функции:

1. Выводить на экран в двоичной форме следующие регистры контроллеров прерывания (как ведущего, так и ведомого):
   - регистр запросов на прерывания;
   - регистр обслуживаемых прерываний;
   - регистр масок.

   При этом значения регистров должны выводиться всегда в одно и то же место экрана.
2. Осуществлять переход на стандартные обработчики аппаратных прерываний, для обеспечения нормальной работы компьютера.

Алгоритм решения задачи:

- Все векторы аппаратных прерываний ведущего и ведомого контроллера переносятся на пользовательские прерывания с помощью функций getvect и setvect.
- Производится инициализация контроллеров, заключающаяся в после-довательности команд: ICW1, ICW2, ICW3 и ICW4.
- С помощью функции _dos_keep осуществляется выход в DOS, при этом программа остаётся резидентной.
- В каждом обработчике выводятся в видеопамять в двоичной форме значения регистров запросов на прерывания, обслуживаемых прерываний, ма-сок. Затем вызываются стандартные обработчики прерываний.

Листинг программы:

Далее приведен листинг резидентной программы, выполняющей перенос всех векторов аппаратных прерываний ведущего и ведомого контроллера на поль-зовательские прерывания.

```
#include <dos.h>
#include <stdio.h>
#include <stdlib.h>

struct VIDEO {
    unsigned char symbol;
    unsigned char attribute;
};

void print(void);
void initialize(void);
```

```
// old_handlers
void interrupt (*old_int08)(void);
void interrupt (*old_int09)(void);
void interrupt (*old_int0A)(void);
void interrupt (*old_int0B)(void);
void interrupt (*old_int0C)(void);
void interrupt (*old_int0D)(void);
void interrupt (*old_int0E)(void);
void interrupt (*old_int0F)(void);

void interrupt (*old_int70)(void);
void interrupt (*old_int71)(void);
void interrupt (*old_int72)(void);
void interrupt (*old_int73)(void);
void interrupt (*old_int74)(void);
void interrupt (*old_int75)(void);
void interrupt (*old_int76)(void);
void interrupt (*old_int77)(void);


// new handlers for vectors 08h-0Fh (Master)
void interrupt new_int08(void); // IRQ0 (08h)
void interrupt new_int09(void); // IRQ1 (09h)
void interrupt new_int0A(void); // IRQ2 (0Ah)
void interrupt new_int0B(void); // IRQ3 (0Bh)
void interrupt new_int0C(void); // IRQ4 (0Ch)
void interrupt new_int0D(void); // IRQ5 (0Dh)
void interrupt new_int0E(void); // IRQ6 (0Eh)
void interrupt new_int0F(void); // IRQ7 (0Fh)

// new handlers for vectors 78h - 7Fh (Slave)
void interrupt new_int78(void); // IRQ8 (78h)
void interrupt new_int79(void); // IRQ9 (79h)
void interrupt new_int7A(void); // IRQ10 (7Ah)
void interrupt new_int7B(void); // IRQ11 (7Bh)
void interrupt new_int7C(void); // IRQ12 (7Ch)
void interrupt new_int7D(void); // IRQ13 (7Dh)
void interrupt new_int7E(void); // IRQ14 (7Eh)
void interrupt new_int7F(void); // IRQ15 (7Fh)


void print()
{
    unsigned char color = 0x0F;
    unsigned char master_mask, slave_mask;
    unsigned char master_request, slave_request;
    unsigned char master_service, slave_service;
    int x_start = 10;
    int y_mask = 1;
    int y_request = 2;
    int y_service = 3;
    int i;
    unsigned char val;
    unsigned char mask;
    struct VIDEO far* p;

    master_mask = inp(0x21);
    slave_mask = inp(0xA1);
    outp(0x20, 0x0A);
    master_request = inp(0x20);
    outp(0xA0, 0x0A);
    slave_request = inp(0xA0);
    outp(0x20, 0x0B);
    master_service = inp(0x20);
```

```
        outp(0xA0, 0x0B);
        slave_service = inp(0xA0);

        // MASK output
        p = (struct VIDEO far*) MK_FP(0xB800, (y_mask * 80 + x_start) * 2);
        mask = 0x80;
        for (i = 0; i < 8; i++) {
            p->symbol = (master_mask & mask) ? '1' : '0';
            p->attribute = color;
            p++;
            mask >>= 1;
        }
        p->symbol = ' ';
        p->attribute = color;
        p++;
        mask = 0x80;
        for (i = 0; i < 8; i++) {
            p->symbol = (slave_mask & mask) ? '1' : '0';
            p->attribute = color;
            p++;
            mask >>= 1;
        }

        // REQUEST output
        p = (struct VIDEO far*) MK_FP(0xB800, (y_request * 80 + x_start) *
2);
        mask = 0x80;
        for (i = 0; i < 8; i++) {
            p->symbol = (master_request & mask) ? '1' : '0';
            p->attribute = color;
            p++;
            mask >>= 1;
        }
        p->symbol = ' ';
        p->attribute = color;
        p++;
        mask = 0x80;
        for (i = 0; i < 8; i++) {
            p->symbol = (slave_request & mask) ? '1' : '0';
            p->attribute = color;
            p++;
            mask >>= 1;
        }

        // SERVICE output
        p = (struct VIDEO far*) MK_FP(0xB800, (y_service * 80 + x_start) *
2);
        mask = 0x80;
        for (i = 0; i < 8; i++) {
            p->symbol = (master_service & mask) ? '1' : '0';
            p->attribute = color;
            p++;
            mask >>= 1;
        }
        p->symbol = ' ';
        p->attribute = color;
        p++;
        mask = 0x80;
        for (i = 0; i < 8; i++) {
            p->symbol = (slave_service & mask) ? '1' : '0';
            p->attribute = color;
            p++;
            mask >>= 1;
        }
```

```
}


// new handlers for vectors 08h-0Fh (Master)
void interrupt new_int08(void) { print(); old_int08(); } // IRQ0 (08h)
void interrupt new_int09(void) { print(); old_int09(); } // IRQ1 (09h)
void interrupt new_int0A(void) { print(); old_int0A(); } // IRQ2 (0Ah)
void interrupt new_int0B(void) { print(); old_int0B(); } // IRQ3 (0Bh)
void interrupt new_int0C(void) { print(); old_int0C(); } // IRQ4 (0Ch)
void interrupt new_int0D(void) { print(); old_int0D(); } // IRQ5 (0Dh)
void interrupt new_int0E(void) { print(); old_int0E(); } // IRQ6 (0Eh)
void interrupt new_int0F(void) { print(); old_int0F(); } // IRQ7 (0Fh)

// new handlers for vectors 78h - 7Fh (Slave)
void interrupt new_int78(void) { print(); old_int70(); } // IRQ8 (78h)
void interrupt new_int79(void) { print(); old_int71(); } // IRQ9 (79h)
void interrupt new_int7A(void) { print(); old_int72(); } // IRQ10 (7Ah)
void interrupt new_int7B(void) { print(); old_int73(); } // IRQ11 (7Bh)
void interrupt new_int7C(void) { print(); old_int74(); } // IRQ12 (7Ch)
void interrupt new_int7D(void) { print(); old_int75(); } // IRQ13 (7Dh)
void interrupt new_int7E(void) { print(); old_int76(); } // IRQ14 (7Eh)
void interrupt new_int7F(void) { print(); old_int77(); } // IRQ15 (7Fh)


void initialize()
{
    old_int08 = getvect(0x08);
    old_int09 = getvect(0x09);
    old_int0A = getvect(0x0A);
    old_int0B = getvect(0x0B);
    old_int0C = getvect(0x0C);
    old_int0D = getvect(0x0D);
    old_int0E = getvect(0x0E);
    old_int0F = getvect(0x0F);

    old_int70 = getvect(0x70);
    old_int71 = getvect(0x71);
    old_int72 = getvect(0x72);
    old_int73 = getvect(0x73);
    old_int74 = getvect(0x74);
    old_int75 = getvect(0x75);
    old_int76 = getvect(0x76);
    old_int77 = getvect(0x77);

    setvect(0x08, new_int08);
    setvect(0x09, new_int09);
    setvect(0x0A, new_int0A);
    setvect(0x0B, new_int0B);
    setvect(0x0C, new_int0C);
    setvect(0x0D, new_int0D);
    setvect(0x0E, new_int0E);
    setvect(0x0F, new_int0F);

    setvect(0x78, new_int78);
    setvect(0x79, new_int79);
    setvect(0x7A, new_int7A);
    setvect(0x7B, new_int7B);
    setvect(0x7C, new_int7C);
    setvect(0x7D, new_int7D);
    setvect(0x7E, new_int7E);
    setvect(0x7F, new_int7F);

    _disable();
```

```
    outp(0x20, 0x11);
    outp(0x21, 0x08);
    outp(0x21, 0x04);
    outp(0x21, 0x01);

    outp(0xA0, 0x11);
    outp(0xA1, 0x78);
    outp(0xA1, 0x02);
    outp(0xA1, 0x01);

    _enable();

}


int main()
{
    unsigned far *p;

    initialize();
    system("cls");

    printf("MASTER    SLAVE\n");
    printf("MASK:                 \n");
    printf("REQUEST:              \n");
    printf("SERVICE:              \n");

    FP_SEG(p) = _psp;
    FP_OFF(p) = 0x2C;
    _dos_freemem(*p);

    _dos_keep(0, (_DS - _CS) + (_SP / 16) + 1);
    return 0;
}
```

Тестовые примеры

Рисунок 4.1 – Результат работы программы при запуске.

Рисунок 4.2 – Результат работы программы при нажатии кнопки.

Заключение

В ходе лабораторной удалось выполнить перенос всех векторов аппаратных прерываний ведущего и ведомого контроллера на пользовательские прерывания. Использование контроллера прерываний позволяет ускорить взаимодействие процессора с внешними устройствами. Недостатком программы является клонирование программы в памяти при повторном запуске.

Программа компилировалась в Turbo C++ и запускалась в DOS, который эмулировался с помощью DosBox.